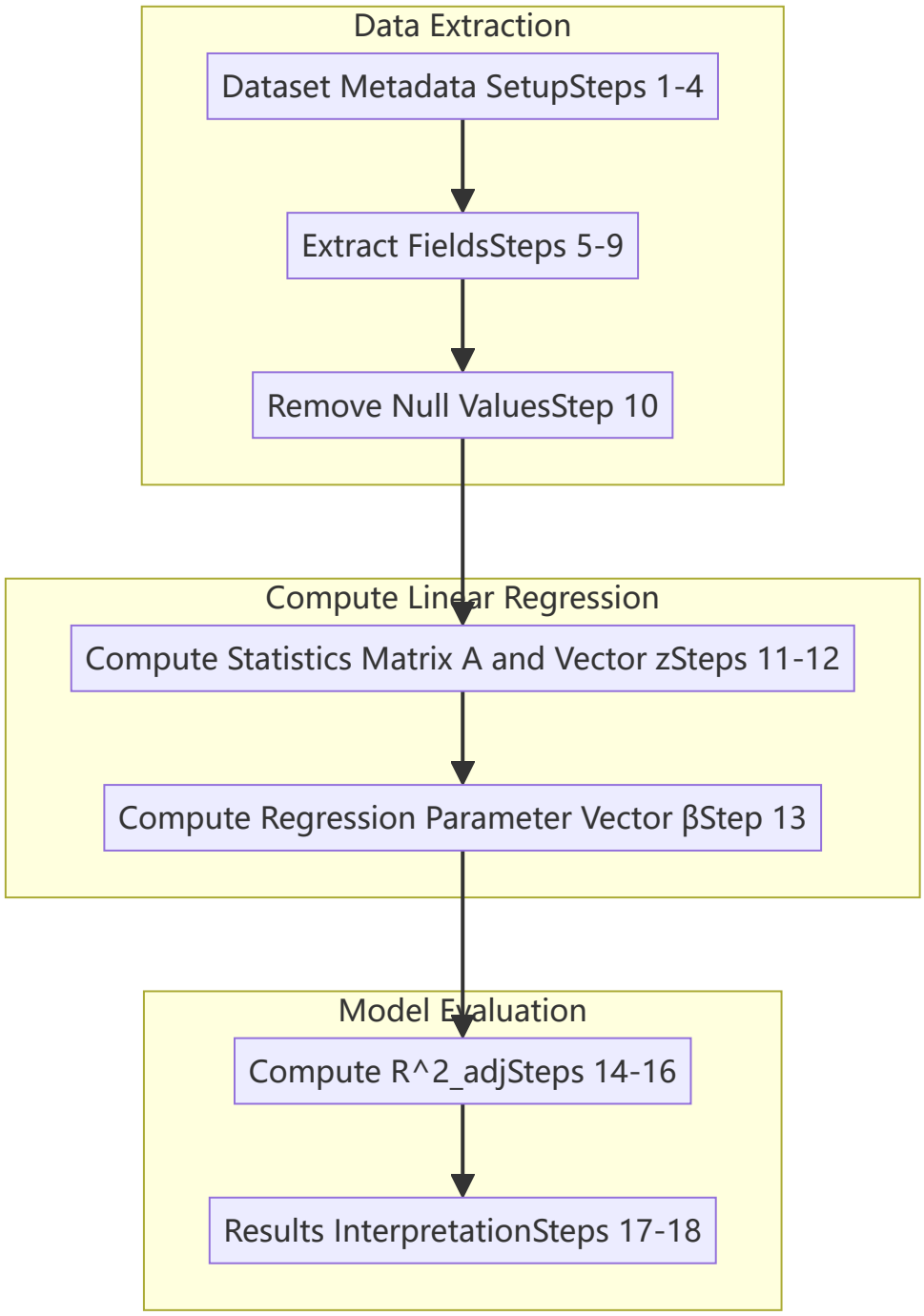
Multiple Linear Regression - Predicting Health Status from Demographic Data

In this assignment, you will analyze data from the Behavioral Risk Factor Surveillance System (BRFSS) data. The objective is to construct a self-contained Jupyter Notebook that predicts health outcomes using income, body mass index (BMI), and education as explanatory variables. Importantly, your solution should be designed to handle the dataset without loading all data into memory simultaneously.

Before beginning, carefully

1. Review the Lecture Slides on Multiple Linear Regression, specifically:

    - Slides 27–31: Derivation of the formula for multiple linear regression coefficients $\hat{\beta} = (X^T X)^{-1} X^T y$.
    - Slides 36–58: A step-by-step algorithm for computing the regression coefficients and the adjusted R-squared statistic in a single pass over the data.



2. Review the Provided Script Template.

   - Examine the sample Python script template in `linearRegression.zip`
   - Download the BRFSS data files for years 2011, 2012, 2013, and 2014 into the `./data` directory. Do not unzip the downloaded files.
   - Complete the missing sections of the script. You are required to implement a total of **four lines** of code.
   - Execute the completed script to compute the regression coefficients and the adjusted R-squared statistic by running the following command in your terminal:
     `python -m ScalableAlgorithms.PythonScripts.linearRegression`

Your task:

- Adapt the provided Python script template to implement a function `regression_health` with the following specifications:
  - INPUT:
    - `data_dir`: Path to the directory containing the BRFSS data files (the zip files should remain compressed).
    - `columns`: A non-empty list of column names to be used as predictors (independent variables).
  - OUTPUT:
    - `beta`: A NumPy array of regression coefficients. The first element corresponds to the intercept $\hat{\beta}_0$, followed by the coefficients for each predictor in the order they are listed in `columns`.
    - `adj_r2`: The adjusted R-squared statistic of the regression model.

Tool Functions

In [ ]:
```
import sys
import os
import importlib
import pathlib
import numpy as np
import zipfile
from io import TextIOWrapper
```

In [ ]:
```
def fieldDictBuild():
    fieldDict = dict.fromkeys([0, 1, 2, 3, 11, 12, 13, 14])

    fieldDict[11] = {
        "genhlth": 73,
        "bmi": (1533, 1536),
        "income": (124, 125),
        "education": 122,
    }
    fieldDict[12] = {
        "genhlth": 73,
        "bmi": (1644, 1647),
        "income": (116, 117),
        "education": 114,
    }
    fieldDict[13] = {
        "genhlth": 80,
        "bmi": (2192, 2195),
        "income": (152, 153),
        "education": 150,
    }
    fieldDict[14] = {
        "genhlth": 80,
        "bmi": (2247, 2250),
        "income": (152, 153),
        "education": 150,
    }

    return fieldDict

def getIncome(incomeString):
```

```
    if incomeString != "  ":
        income = int(incomeString)
    else:
        income = 9
    return income

def convertBMI(bmiString, shortYear):
    bmi = 0
    if shortYear == 0 and bmiString != "999":
        bmi = 0.1 * float(bmiString)
    if shortYear == 1 and bmiString != "999999":
        bmi = 0.0001 * float(bmiString)
    if 2 <= shortYear <= 10 and bmiString != "9999":
        bmi = 0.01 * float(bmiString)
    if shortYear > 10 and bmiString != "    ":
        bmi = 0.01 * float(bmiString)
    return bmi

def getEducation(educationString):
    if educationString != " ":
        education = int(educationString)
    else:
        education = 9
    return education

def getHlth(hlthString):
    if hlthString != " ":
        genhlth = int(hlthString)
        if genhlth > 6:
            genhlth = -1
    else:
        genhlth = -1

    assert genhlth in (-1, 1, 2, 3, 4, 5, 6)
    return genhlth
```

Main Regression Function

In [ ]:

```
def regression_health(data_dir, columns):
    """
    Perform linear regression on health data.
    IN: data_dir, str, directory containing data files
    IN: columns, list of str, column names to use as features
    OUT: beta, np.array of shape (len(columns) + 1,), regression coefficients
    OUT: adj_r2, float, adjusted R-squared of the regression
    """

    # Adapt linearRegression.py here
    # YOUR CODE HERE



    return beta, adj_r2
```

In [ ]:

```
if __name__ == "__main__":
    beta_full, r2_full = regression_health('./data', ['education', 'income', 'bm
    print(beta_full, r2_full)
```